



Politécnico
Internacional

ESTRUCTURA DE DATOS



**“No me importa si funciona en su máquina!
No me envían su máquina!.”**

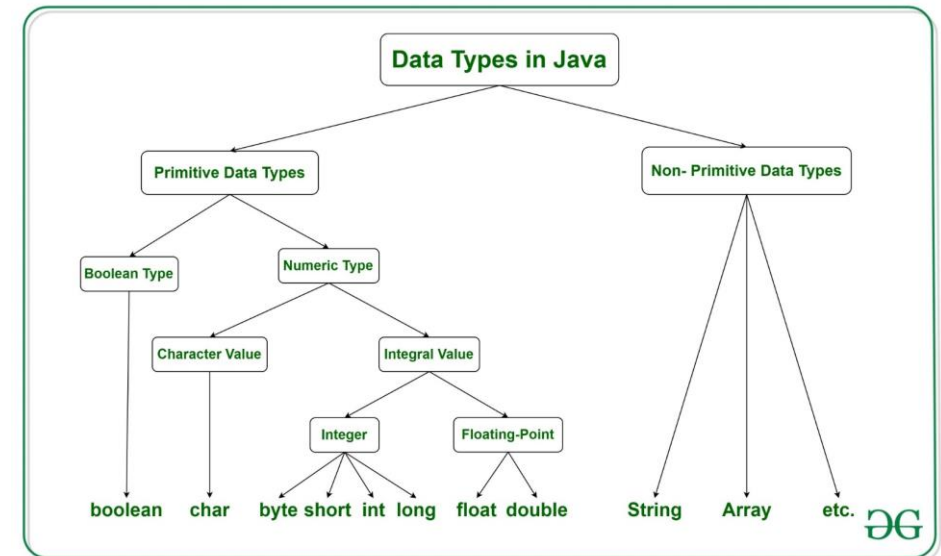
VIDIU PLATON

AGENDA

- Conceptos Java
- Innova Colombia
- Conclusiones

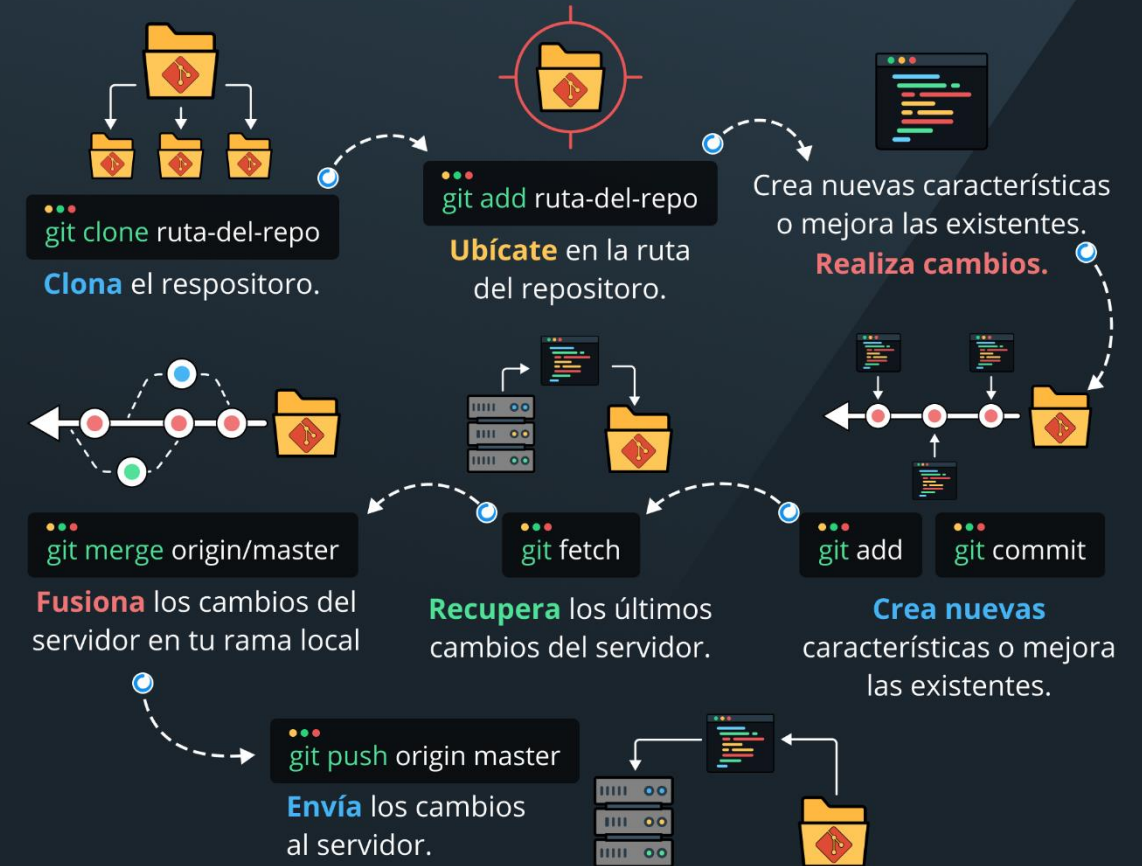
Retroalimentación

- ¿Qué es Java?
- Características – Seguridad
- POO
- Sintaxis Básica
- Historia
- Tipado – Estático & Dinámico
- Tipos de datos.





¿CÓMO CONTRIBUIR EN UN PROYECTO CON GIT?



Aprende a controlar las versiones de tus proyectos:

 ed.team/cursos/git



TÉRMINOS DE git QUE DEBES CONOCER

REPOSITORY/REPO

Base de datos donde se almacena el historial del código.

COMMIT

Registro de uno o varios cambios hechos en el repositorio.

STAGE

Lista de archivos que se usarán para un commit.

FORK

Copia de un repositorio.

BRANCH / RAMA

Entorno o espacio de trabajo independiente en Git.

MASTER

Rama principal de un repositorio.

CHECKOUT

Es la acción de **moverse entre diferentes ramas**.

MERGE

Combina dos o más ramas en una.

Si eres programador es **obligatorio** aprender GIT. Domínalo en:

 ed.team/cursos/git



¿QUÉ ES LA PROGRAMACIÓN ORIENTADA A OBJETOS?

- Los objetos se crean a partir de una plantilla llamada **clase**. Cada objeto es una instancia de su clase.



CLASE

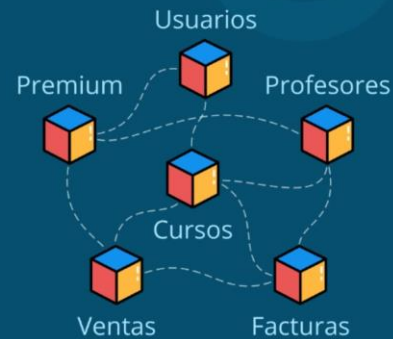
INSTANCIACIÓN

OBJETO



- Los objetos tienen **datos (atributos)** y **funcionalidades (métodos)**.

- En una aplicación los objetos están separados **pero se comunican entre ellos**.



ATRIBUTOS

Nombres
Apellidos
Correo
Contraseña
Premium



MÉTODOS

Editar perfil
Iniciar sesión
Cerrar sesión
Cambiar contraseña
Pasar a premium



Puedes programar con este paradigma **en la mayoría de lenguajes**.



Aprende a crear aplicaciones usando la POO en:

 ed.team/cursos

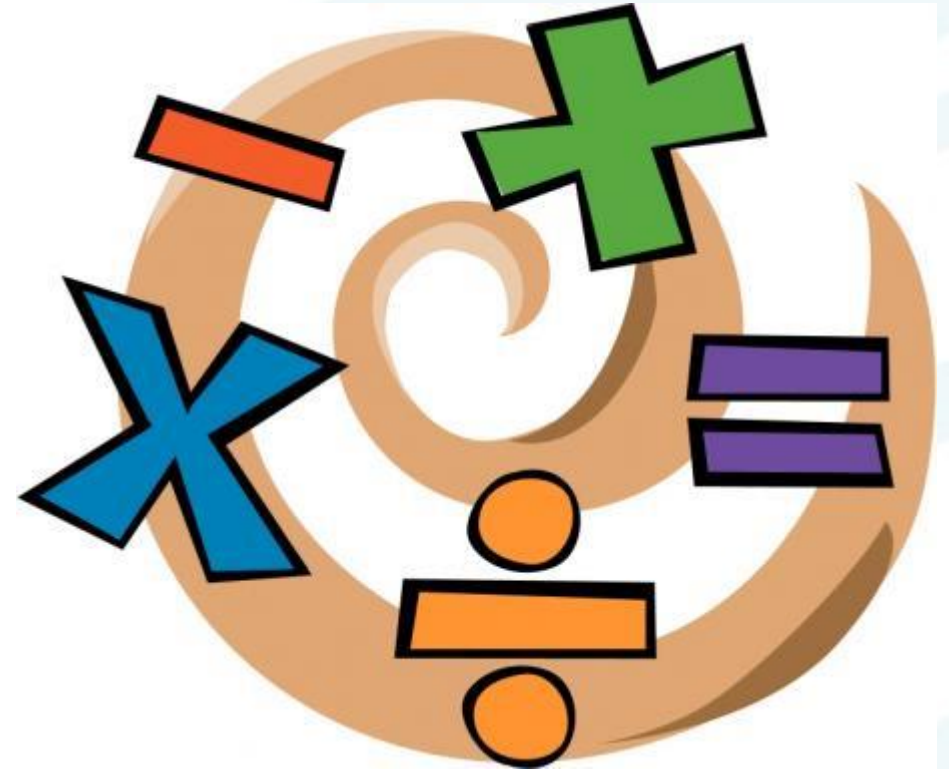
Es un **paradigma de programación** que organiza las funciones en entidades llamadas objetos.



CLASIFICACION OPERADORES JAVA

Java proporciona muchos tipos de operadores que se pueden usar según la necesidad. Se clasifican según la funcionalidad que brindan. Algunos de los tipos son los siguientes:

- *Operadores aritméticos*
- *Operadores unarios*
- *Operador de asignación*





OPERADORES ARITMÉTICOS

DATOS PRIMITIVOS

JAVA + OPERADORES ARITMÉTICOS

```
// Programa Java para ilustrar
// operadores aritméticos
public class operators
{
    public static void main(String[] args)
    {
        int a = 20, b = 10, c = 0, d = 20, e = 40, f = 30;
        String x = "Thank", y = "You";

        // Operador + y -
        System.out.println("a + b = " + (a + b));
        System.out.println("a - b = " + (a - b));

        // El operador + si se usa con strings
        // concatena las cadenas dadas.
        System.out.println("x + y = " + x + y);

        // Operador * y /
        System.out.println("a * b = " + (a * b));
        System.out.println("a / b = " + (a / b));

        // operador de módulo da el resto
        // de dividir el primer operando con el segundo
        System.out.println("a % b = " + (a % b));

        // si el denominador es 0 en la división
        // System.out.println(a/c);
        // lanzaría una java.lang.ArithmeticException

    }
}
```

Se utilizan para realizar operaciones aritméticas simples en tipos de datos primitivos.

**: Multiplicación*

/: División

?: Módulo

+: Adición

-: Resta

OPERADORES UNARIOS

DATOS PRIMITIVOS

JAVA + OPERADORES UNARIOS

```
// Programa Java para ilustrar
// operadores unarios
public class operators
{
    public static void main(String[] args)
    {
        int a = 20, b = 10, c = 0, d = 20, e = 40, f = 30;
        boolean condition = true;

        // operador de pre-incremento
        // a = a+1 y entonces c = a;
        c = ++a;
        System.out.println("Valor de c (++a) = " + c);

        // operador de post-incremento
        // c=b entonces b=b+1 (b pasa a ser 11)
        c = b++;
        System.out.println("Valor de c (b++) = " + c);

        // operador de pre-decremento
        // d=d-1 entonces c=d
        c = --d;
        System.out.println("Valor de c (--d) = " + c);

        // operador de post-decremento
        // c=e entonces e=e-1 (e pasa a ser 39)
        c = e--;
        System.out.println("Valor de c (e--) = " + c);

        // Operador lógico not
        System.out.println("Valor de !condition = " + !condition);
    }
}
```

Los operadores unarios solo necesitan un operando. Se usan para incrementar, disminuir o negar un valor.

-: Unario menos, utilizado para negar los valores.

+: Unario más, usado para dar valores positivos. Solo se usa cuando se convierte deliberadamente un valor negativo en positivo.

++: Operador de incremento, utilizado para incrementar el valor en 1. Hay dos variedades de operador de incremento.

Pre-Incremento: el valor se incrementa primero y luego se calcula el resultado.

OPERADORES UNARIOS

DATOS PRIMITIVOS

JAVA + OPERADORES UNARIOS

```
// Programa Java para ilustrar
// operadores unarios
public class operators
{
    public static void main(String[] args)
    {
        int a = 20, b = 10, c = 0, d = 20, e = 40, f = 30;
        boolean condition = true;

        // operador de pre-incremento
        // a = a+1 y entonces c = a;
        c = ++a;
        System.out.println("Valor de c (++a) = " + c);

        // operador de post-incremento
        // c=b entonces b=b+1 (b pasa a ser 11)
        c = b++;
        System.out.println("Valor de c (b++) = " + c);

        // operador de pre-decremento
        // d=d-1 entonces c=d
        c = --d;
        System.out.println("Valor de c (--d) = " + c);

        // operador de post-decremento
        // c=e entonces e=e-1 (e pasa a ser 39)
        c = e--;
        System.out.println("Valor de c (e--) = " + c);

        // Operador lógico not
        System.out.println("Valor de !condition = " + !condition);
    }
}
```

Los operadores unarios solo necesitan un operando. Se usan para incrementar, disminuir o negar un valor.

Post-Incremento: el valor se usa por primera vez para calcular el resultado y luego se incrementa.

—: Operador de decremento , usado para incrementar el valor en 1. Hay dos variedades de operador de incremento.

Pre-Decremento: el valor se disminuye primero y luego se calcula el resultado.

Post-Decremento: el valor se usa por primera vez para calcular el resultado y luego se disminuye.

! : Operador lógico “no”, utilizado para invertir un valor booleano.

OPERADORES ASIGNACIÓN

DATOS PRIMITIVOS

JAVA + OPERADORES ASIGNACIÓN

El operador de asignación se usa para asignar un valor a cualquier variable. Tiene una asociación de derecha a izquierda, es decir, el valor dado en el lado derecho del operador se asigna a la variable de la izquierda y, por lo tanto, el valor del lado derecho debe declararse antes de usarlo o debe ser una constante.

El formato general del operador de asignación es, `[java]variable = valor;[/java]`

```
// Programa Java para ilustrar
// Operadores de Asignación
public class operators
{
    public static void main(String[] args)
    {
        int a = 20, b = 10, c, d, e = 10, f = 4, g = 9;

        // operador de asignación simple
        c = b;
        System.out.println("Valor de c = " + c);

        // Esta siguiente declaración arrojaría una exception
        // porque el valor del operando derecho debe ser inicializado
        // antes de la asignación, entonces el programa no
        // compila.
        // c = d;

        // operadores de asignación simples
        a = a + 1;
        b = b - 1;
        e = e * 2;
        f = f / 2;
        System.out.println("a,b,e,f = " + a + ","
                           + b + "," + e + "," + f);

        a = a - 1;
        b = b + 1;
        e = e / 2;
        f = f * 2;

        // operados de asignación compuestos/cortos
        a += 1;
        b -= 1;
        e *= 2;
        f /= 2;
        System.out.println("a,b,e,f (usando operadores cortos)= " +
                           a + "," + b + "," + e + "," + f);
    }
}
```

OPERADORES ASIGNACIÓN

DATOS PRIMITIVOS

JAVA + OPERADORES ASIGNACIÓN

*En muchos casos, el operador de asignación se puede combinar con otros operadores para construir una versión más corta de la declaración llamada **Declaración Compuesta** (Compound Statement). Por ejemplo, en lugar de $a = a + 5$, podemos escribir $a += 5$.*

*$int a = 5;$
 $a += 5; // a = a + 5;$*

```
// Programa Java para ilustrar
// Operadores de Asignación
public class operators
{
    public static void main(String[] args)
    {
        int a = 20, b = 10, c, d, e = 10, f = 4, g = 9;

        // operador de asignación simple
        c = b;
        System.out.println("Valor de c = " + c);

        // Esta siguiente declaración arrojaría una exception
        // porque el valor del operando derecho debe ser inicializado
        // antes de la asignación, entonces el programa no
        // compila.
        // c = d;

        // operadores de asignación simples
        a = a + 1;
        b = b - 1;
        e = e * 2;
        f = f / 2;
        System.out.println("a,b,e,f = " + a + ", "
                           + b + ", " + e + ", " + f);

        a = a - 1;
        b = b + 1;
        e = e / 2;
        f = f * 2;

        // operados de asignación compuestos/cortos
        a += 1;
        b -= 1;
        e *= 2;
        f /= 2;
        System.out.println("a,b,e,f (usando operadores cortos)= " +
                           a + ", " + b + ", " + e + ", " + f);
    }
}
```

OPERADORES ASIGNACIÓN

DATOS PRIMITIVOS

JAVA + OPERADORES ASIGNACIÓN

```
// Programa Java para ilustrar
// Operadores de Asignación
public class operators
{
    public static void main(String[] args)
    {
        int a = 20, b = 10, c, d, e = 10, f = 4, g = 9;

        // operador de asignación simple
        c = b;
        System.out.println("Valor de c = " + c);

        // Esta siguiente declaración arrojaría una exception
        // porque el valor del operando derecho debe ser inicializado
        // antes de la asignación, entonces el programa no
        // compila.
        // c = d;

        // operadores de asignación simples
        a = a + 1;
        b = b - 1;
        e = e * 2;
        f = f / 2;
        System.out.println("a,b,e,f = " + a + ","
                           + b + "," + e + "," + f);

        a = a - 1;
        b = b + 1;
        e = e / 2;
        f = f * 2;

        // operados de asignación compuestos/cortos
        a += 1;
        b -= 1;
        e *= 2;
        f /= 2;
        System.out.println("a,b,e,f (usando operadores cortos)= " +
                           a + "," + b + "," + e + "," + f);
    }
}
```

+ = , para sumar el operando izquierdo con el operando derecho y luego asignarlo a la variable de la izquierda.

- = , para restar el operando izquierdo con el operando derecho y luego asignarlo a la variable de la izquierda.

- * = , para multiplicar el operando izquierdo con el operando derecho y luego asignándolo a la variable de la izquierda.*



OPERADORES ASIGNACIÓN

DATOS PRIMITIVOS

JAVA + OPERADORES ASIGNACIÓN

```
// Programa Java para ilustrar
// Operadores de Asignación
public class operators
{
    public static void main(String[] args)
    {
        int a = 20, b = 10, c, d, e = 10, f = 4, g = 9;

        // operador de asignación simple
        c = b;
        System.out.println("Valor de c = " + c);

        // Esta siguiente declaración arrojaría una exception
        // porque el valor del operando derecho debe ser inicializado
        // antes de la asignación, entonces el programa no
        // compila.
        // c = d;

        // operadores de asignación simples
        a = a + 1;
        b = b - 1;
        e = e * 2;
        f = f / 2;
        System.out.println("a,b,e,f = " + a + ", "
                           + b + ", " + e + ", " + f);

        a = a - 1;
        b = b + 1;
        e = e / 2;
        f = f * 2;

        // operados de asignación compuestos/cortos
        a += 1;
        b -= 1;
        e *= 2;
        f /= 2;
        System.out.println("a,b,e,f (usando operadores cortos)= " +
                           a + ", " + b + ", " + e + ", " + f);
    }
}
```

/ = , para dividir el operando izquierdo con el operando derecho y luego asignarlo a la variable de la izquierda.

^ = , para aumentar la potencia del operando izquierdo al operando derecho y asignarlo a la variable de la izquierda.

% = , para asignar el módulo del operando izquierdo con el operando derecho y luego asignarlo a la variable de la izquierda.



PALABRAS RESERVADAS



En los lenguajes de programación, los identificadores (como su nombre lo indica) se utilizan con fines de identificación. En Java, un identificador puede ser un nombre de clase, un nombre de método o un nombre de variable.

PALABRAS RESERVADAS

```
public class Test
{
    public static void main(String[] args)
    {
        int a = 28;
    }
}
```



IDENTIFICADORES

***TEST:** nombre de clase.*

***MAIN:** nombre del método.*

***STRING:** nombre de clase
predefinido.*

***ARGS:** nombre de la variable.*

***A:** nombre de la variable.*



REGLAS IDENTIFICADORES JAVA

JAVA

```
void createFile(final SyntaxNode sn) throws CodeException {
    for (Iterator<SyntaxNode> ite=sn.getChildren().iterator(); ite.hasNext(); ite.next()) {
        final SyntaxNode child = (SyntaxNode) ite.next();
        final Rule rule = child.getRule();
        if (rule.isPackage() || rule.isImport()) {
            // TODO handle static and local
            final SyntaxNode sn = child.getChildByRule(RULE_IMPORT);
            final String fullName = ccn.getTokenChars(sn);
            final String[] parts = fullName.split("\\.");
        }
    }
}
```



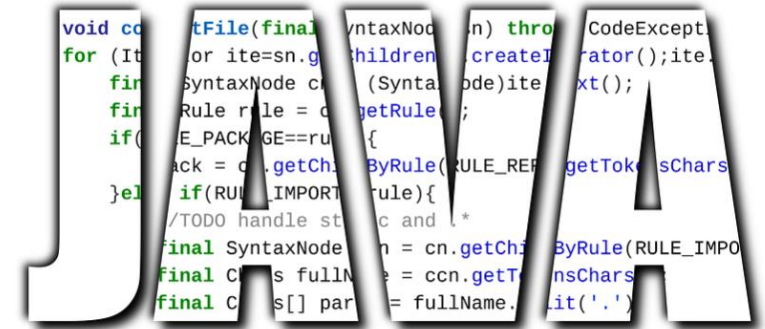
Los únicos caracteres permitidos para los identificadores son todos los caracteres alfanuméricos ([AZ], [az], [0-9]), “\$” (signo de dólar) y ‘_’ (guión bajo). Por ejemplo, “java@” no es un identificador de Java válido ya que contiene “@” – carácter especial.

REGLAS IDENTIFICADORES JAVA



Los identificadores no deben comenzar con dígitos ([0-9]). Por ejemplo, “123java” no es un identificador de Java válido.

Los identificadores de Java distinguen entre mayúsculas y minúsculas.



```
void compileFile(final SyntaxNode n) throws CodeException {
    for (Iterator<SyntaxNode> ite = n.getChildren().iterator(); ite.hasNext(); ite = ite.next()) {
        final SyntaxNode child = (SyntaxNode) ite.next();
        Rule rule = child.getRule();
        if (rule == null) {
            // Handle package name
            String pack = child.getChars().toString();
            if (rule == null) {
                // Handle static and *
                final SyntaxNode n = child.getChars().toString();
                final Char fullN = ccn.getChars().toString();
                final Char[] parts = fullN.split('.');
            }
        }
    }
}
```



REGLAS IDENTIFICADORES JAVA

J A V A

```
void createFile(final SyntaxNode sn) throws CodeException {
    for (Iterator<SyntaxNode> ite=sn.getChildren().iterator(); ite.hasNext(); ite.next()) {
        final SyntaxNode child = (SyntaxNode) ite.next();
        final Rule rule = child.getRule();
        if (rule.getPackageName() != null) {
            final String pack = child.getCharsByRule(RULE_REFERENCE.getTokensChars());
        } else if (rule.isImport()) {
            //TODO handle static and *
            final SyntaxNode n = child.getCharsByRule(RULE_IMPORT.getTokensChars());
            final String fullName = child.getTokensChars();
            final String[] parts = fullName.split("\\.");
        }
    }
}
```



No hay límite en la longitud del identificador, pero es aconsejable usar solamente una longitud óptima de 4 a 15 caracteres.

EJEMPLOS IDENTIFICADORES JAVA

BIEN DEFINIDOS

```
MyVariable  
MYVARIABLE  
myvariable  
x  
i  
x1  
i1  
_myvariable  
$myvariable  
sum_of_array  
javadesdecero
```

MAL DEFINIDOS

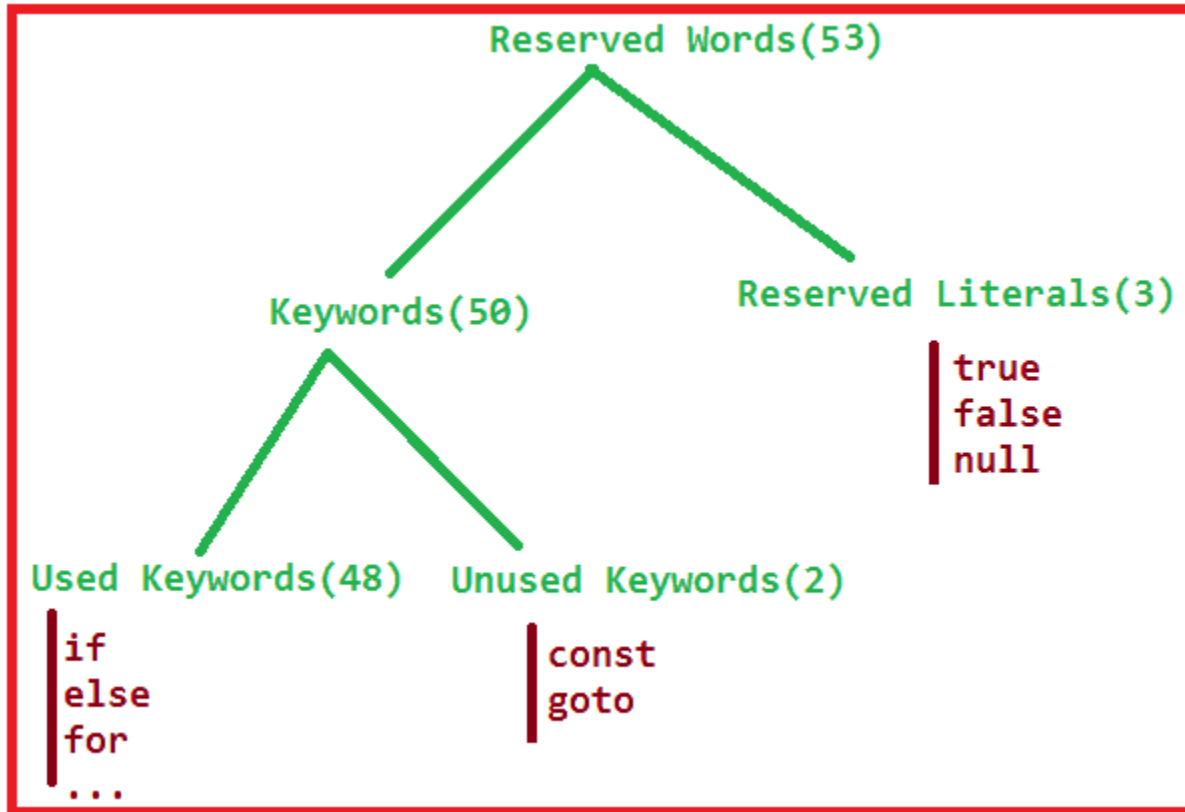
```
My Variable // Contiene un espacio  
123java // Comienza con un dígito  
a+c // El signo más (+) no es un carácter alfanumérico  
variable-2 // El guión (-) no es un carácter alfanumérico  
sum_&_difference // ampersand (&) no es un carácter válido
```



PALABRAS RESERVADAS EN JAVA



Cualquier lenguaje de programación reserva algunas palabras para representar funcionalidades definidas por ese lenguaje. Estas palabras se llaman palabras reservadas. Estas pueden ser categorizadas brevemente en dos partes: palabras clave/keywords (50) y literales (3).



PALABRAS RESERVADAS EN JAVA

Las palabras clave definen funcionalidades y las literales definen un valor.

PALABRAS RESERVADAS EN JAVA

Tabla de Palabras Claves Reservadas en Java

abstract	assert	boolean	break	byte	case	catch	char	class	const	continue
default	do	double	else	enum	extends	false	final	finally	float	for
goto	if	implements	import	instanceof	int	interface	long	native	new	null
package	private	protected	public	return	short	static	strictfp	String	super	switch
synchronized	this	throw	throws	transient	true	try	void	volatile	while	



CONVENCIONES DE NOMENCLATURA JAVA

Las convenciones de nombres para el lenguaje de programación Java. Deben seguirse mientras se desarrolla software en Java para un buen mantenimiento y legibilidad del código. Java utiliza CamelCase como una práctica para escribir nombres de métodos, variables, clases, paquetes y constantes.

CLASES E INTERFACES EN JAVA

```
Interface Bicycle  
Class MountainBike implements Bicycle
```

```
Interface Sport  
Class Football implements Sport
```

CONVENCIONES DE NOMENCLATURA JAVA



Los nombres de las clases deben ser sustantivos, en mayúsculas y minúsculas, con la primera letra de cada palabra interna en mayúscula. El nombre de las interfaces también debe estar en mayúscula (la primera) al igual que los nombres de las clases. Use palabras completas y debe evitar acrónimos y abreviaturas.

METODOS EN JAVA

```
void changeGear(int newValue);  
void speedUp(int increment);  
void applyBrakes(int decrement);
```

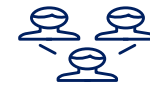
CONVENCIONES DE NOMENCLATURA JAVA



Los métodos deben ser verbos, en mayúsculas y minúsculas, con la primera letra de cada palabra interna (a partir de la segunda) en mayúscula.

VARIABLES EN JAVA

```
// variables para la clase MountainBike  
int speed = 0;  
int gear = 1;
```



CONVENCIONES DE NOMENCLATURA JAVA

Los nombres de las variables deben ser cortos pero significativos.

No debería comenzar con un guión bajo ('_') o caracteres, como por ejemplo, un signo de dólar '\$'.

Debe ser mnemotécnico, es decir, diseñado para indicar al observador casual la intención de su uso.

Se deben evitar los nombres de variable de un carácter, excepto para variables temporales.

Los nombres comunes para las variables temporales son: i, j, k, m y n para enteros; c, d y e para los caracteres.

VARIABLES CONSTANTES EN JAVA

```
static final int MIN_WIDTH = 4;

// Algunas variables constantes utilizadas en la clase Float predefinida
public static final float POSITIVE_INFINITY = 1.0f / 0.0f;
public static final float NEGATIVE_INFINITY = -1.0f / 0.0f;
public static final float NaN = 0.0f / 0.0f;
```



Debería estar todo en mayúsculas con palabras separadas por guiones bajos (“_”).

Hay varias constantes utilizadas en clases predefinidas como Float, Long, String, etc.

CONVENCIONES DE NOMENCLATURA JAVA

PAQUETES EN JAVA

```
com.sun.eng  
com.apple.quicktime.v2  
  
// java.lang packet in JDK  
java.lang
```



CONVENCIONES DE NOMENCLATURA JAVA

El prefijo de un nombre de paquete único siempre se escribe en letras ASCII en minúsculas y debe ser uno de los nombres de dominio de nivel superior, como por ejemplo: com, edu, gov, mil, net, org.

Los componentes posteriores del nombre del paquete varían de acuerdo con las convenciones internas de nombres de la organización.

NOMBRE DE CLASE

0

VARIABLE EN JAVA



CLASES DE PREDIFINIDAS

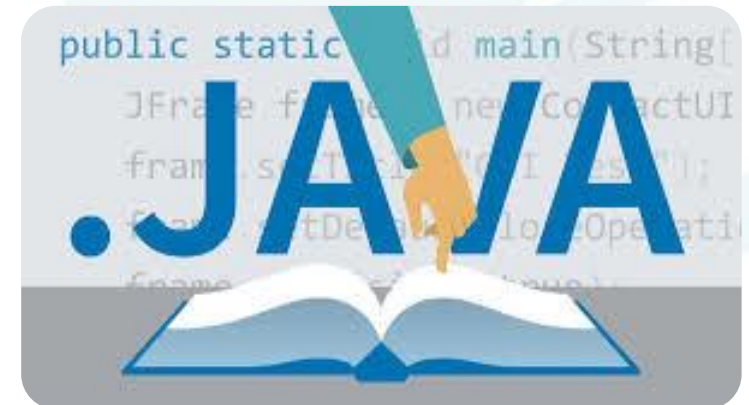
*Los siguientes son algunos nombres de clase **NO VÁLIDOS** en Java*

```
boolean break = false; // no permitido ya que break es palabra clave  
int boolean = 28; // no permitido ya que boolean es palabra clave  
boolean goto = false; // no permitido ya que goto es palabra clave  
String final = "hola"; // no permitido ya que final es palabra clave
```

DECLARACION Y TIPOS DE VARIABLES

Una variable es el nombre dado a una ubicación de memoria. Es la unidad básica de almacenamiento en un programa. El valor almacenado en una variable se puede cambiar durante la ejecución del programa.

NOTA: “En Java, todas las variables deben declararse antes de que puedan ser utilizadas.”



¿CÓMO SE DECLARA VARIABLES?

DATOS PRIMITIVOS

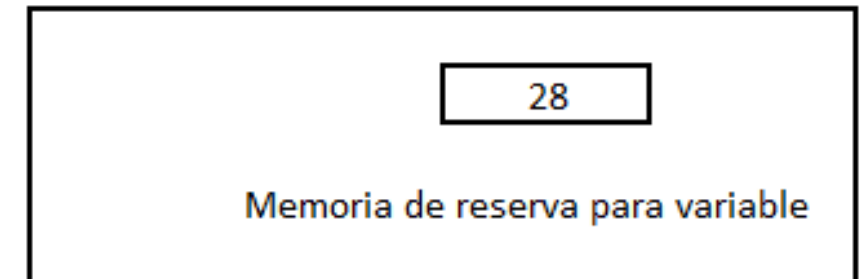
TIPO DE DATO: tipo de datos que se pueden almacenar en esta variable.

NOMBRE_VARIABLE: nombre dado a la variable.

VALOR: es el valor inicial almacenado en la variable.

Diagram illustrating the declaration `int age = 28;`:

- `int`: tipo de dato
- `age`: nombre_variable
- `= 28`: valor



RAM

¿CÓMO DECLARA VARIABLES?

```
float simpleInterest; //Declarando variable float  
int time = 10, speed = 20; //Declarando e Inicializando la variable integer  
char var = 'h'; // Declarando e Inicializando la variable character
```

TIPOS DE VARIABLES

JAVA + TIPOS:

- *VARIABLES LOCALES*
- *VARIABLES DE INSTANCIA*
- *VARIABLES ESTÁTICAS*



Una variable definida dentro de un bloque, método o constructor se llama variable local.

Estas variables se crean cuando el bloque ingresado o método se llama y destruye después de salir del bloque o cuando la llamada regresa del método.

El alcance de estas variables solo existe dentro del bloque en el que se declara la variable, es decir, podemos acceder a estas variables solo dentro de ese bloque.

```
public class StudentDetails
{
    public void StudentAge()
    {    //variable local age
        int age = 0;
        age = age + 5;
        System.out.println("La edad del estudiante es : " + age);
    }

    public static void main(String args[])
    {
        StudentDetails obj = new StudentDetails();
        obj.StudentAge();
    }
}
```

En el programa anterior, la variable age (edad) es una variable local para el método `[java]StudentAge()[/java]`.

Si usamos la variable age fuera del método `[java]StudentAge()[/java]`, el compilador producirá un error como se muestra a continuación en el programa

```
public class StudentDetails
{
    public void StudentAge()
    {    //variable local age
        int age = 0;
        age = age + 5;
        System.out.println("La edad del estudiante es : " + age);
    }

    public static void main(String args[])
    {
        StudentDetails obj = new StudentDetails();
        obj.StudentAge();
    }
}
```

Las variables de instancia son variables no estáticas y se declaran en una clase fuera de cualquier método, constructor o bloque.

Como las variables de instancia se declaran en una clase, estas variables se crean cuando un objeto de la clase se crea y se destruye cuando se destruye el objeto.

A diferencia de las variables locales, podemos usar especificadores de acceso para variables de instancia. Si no especificamos ningún especificador de acceso, se utilizará el especificador de acceso predeterminado.

```
import java.io.*;
class Points
{
    //Estas variables son variables de instancia.
    //Estas variables están en una clase y no están dentro de ninguna función/método
    int engPoints;
    int mathsPoints;
    int phyPoints;
}

class PointsDemo
{
    public static void main(String args[])
    {
        //primer objeto
        Points obj1 = new Points();
        obj1.engPoints = 50;
        obj1.mathsPoints = 80;
        obj1.phyPoints = 90;

        //segundo objeto
        Points obj2 = new Points();
        obj2.engPoints = 80;
        obj2.mathsPoints = 60;
        obj2.phyPoints = 85;

        //mostrando puntos para el primer objeto
        System.out.println("Puntos para el primer objeto:");
        System.out.println(obj1.engPoints);
        System.out.println(obj1.mathsPoints);
        System.out.println(obj1.phyPoints);

        //mostrando puntos para el segundo objeto
        System.out.println("Puntos para el segundo objeto:");
        System.out.println(obj2.engPoints);
        System.out.println(obj2.mathsPoints);
        System.out.println(obj2.phyPoints);
    }
}
```

```
import java.io.*;
class Points
{
    //Estas variables son variables de instancia.
    //Estas variables están en una clase y no están dentro de ninguna función/método
    int engPoints;
    int mathsPoints;
    int phyPoints;
}

class PointsDemo
{
    public static void main(String args[])
    {
        //primer objeto
        Points obj1 = new Points();
        obj1.engPoints = 50;
        obj1.mathsPoints = 80;
        obj1.phyPoints = 90;

        //segundo objeto
        Points obj2 = new Points();
        obj2.engPoints = 80;
        obj2.mathsPoints = 60;
        obj2.phyPoints = 85;

        //mostrando puntos para el primer objeto
        System.out.println("Puntos para el primer objeto:");
        System.out.println(obj1.engPoints);
        System.out.println(obj1.mathsPoints);
        System.out.println(obj1.phyPoints);

        //mostrando puntos para el segundo objeto
        System.out.println("Puntos para el segundo objeto:");
        System.out.println(obj2.engPoints);
        System.out.println(obj2.mathsPoints);
        System.out.println(obj2.phyPoints);
    }
}
```



Las variables estáticas también se conocen como variables de clase.

Estas variables se declaran de forma similar a las variables de instancia, la diferencia es que las variables estáticas se declaran utilizando la palabra clave `[java]static[/java]` dentro de una clase fuera de cualquier constructor o bloque de métodos.

A diferencia de las variables de instancia, solo podemos tener una copia de una variable estática por clase, independientemente de cuántos objetos creemos.

Las variables estáticas se crean al inicio de la ejecución del programa y se destruyen automáticamente cuando finaliza la ejecución.

```
import java.io.*;
class Emp {

    // salario como variable estatica
    public static double salary;
    public static String name = "Alex";
}

public class EmpDemo
{
    public static void main(String args[]) {

        //acceder a la variable estatica sin objeto
        Emp.salary = 1000;
        System.out.println(Emp.name + " tiene un salario promedio de: " + Emp.salary);
    }
}
```

VARIABLES CONSTANTES

Las variables creadas para almacenar valores fijos de esta manera se conocen como “constantes”, y es convencional nombrar constantes con todos los caracteres en mayúsculas, para distinguirlas de las variables regulares. Los programas que intentan cambiar un valor constante no se compilarán, y el compilador javac generará un mensaje de error.

```
class Constants
{
    public static void main ( String[] args ) {
        //inicializamos tres constantes enteras
        final int CONSTANTE1 = 6 ;
        final int CONSTANTE2 = 1 ;
        final int CONSTANTE3 = 3 ;

        //declaramos variables regulares del tipo int
        int td,pat,fg,total;

        //Inicializamos las variables regulares
        td = 4*CONSTANTE1;
        pat = 3*CONSTANTE2;
        fg = 2*CONSTANTE3;

        total = (td+pat+fg) ;

        System.out.println("Resultado: " + total);
    }
}
```

AVANCES **INNOVA** COLOMBIA

¿preguntas?

RETO 1

CONCLUSIONES